FIT3080 Applied Class on Problem Solving as Search: Problem Formulation, Backtrack, Tree Search and Graphsearch

Exercise 1: Backtrack

Specify a state representation, operators and goal test to solve the missionaries and cannibals problem:

Three missionaries and three cannibals come to a river. There is a boat on their side of the river that can be used either by one or two persons. How should they use this boat to cross the river in such a way that cannibals never outnumber missionaries on either side of the river?

(a) Illustrate the workings of the backtrack algorithm (Backtrack or Backtrack1) to solve this problem. To expedite your solution, you may want to specify a heuristic function over the actions for different states.

(b) [Optional] Write a python program to implement your backtrack solution.

Exercise 2: Backtrack, Constraint Satisfaction

Consider a problem where one has to insert the numbers 1, 2, 3, 4, 5 and 6 in the perimeter of the following rectangle, so that each side adds up to 18.

	8		
9	X	X	10
		7	

(a) Express the state space of this problem as a set of constraints on the variables assigned to the different positions in the rectangle, where the following variable assignment is to be used:

x_1	8	x_2	x_3
9	X	X	10
x_4	x_5	7	x_6

(b) Apply algorithm Backtrack (or Backtrack1) to find an assignment of numbers to variables which satisfies the problem statement. You can make only one variable assignment in each step (what will happen if you make more assignments?), but any variable assignment which follows directly from another variable assignment should be performed in the same step.

Number each state created. Indicate clearly which path is active at each point in time, and also indicate dead-end states and the number of the state returned to, if a dead-end state is encountered.

(c) Draw the rectangle with the variable assignment you have found.

Exercise 3: Problem Formulation, Search

You have a 9×9 grid of squares, each of which can be coloured red or blue. The grid is initially coloured all blue, but you can change the colour of any square any number of times. Imagining that the grid is divided into nine 3×3 super-squares, you want each super-square to be all one colour but neighboring super-squares to be different colours.

(a) Formulate this problem in the most straightforward way. Compute the size of the state space. If you were to use a search tree to find a solution, what would be the branching factor including and excluding revisits of ancestor states, and how many states would you generate in a tree of depth d?

(b) Given the goal, we need consider only colourings where each super-square is uniformly coloured. Reformulate the problem and compute the size of the state space. How many solutions does this problem have?

(c) Part (b) has abstracted the original problem (a). Can you give a translation from the solution in (b) into a solution to problem (a)?

Exercise 4: Search Algorithms

(a) Why can you stop BFS when you generate/find the goal, i.e., you don't need to wait for the goal to reach the top of the list in OPEN?

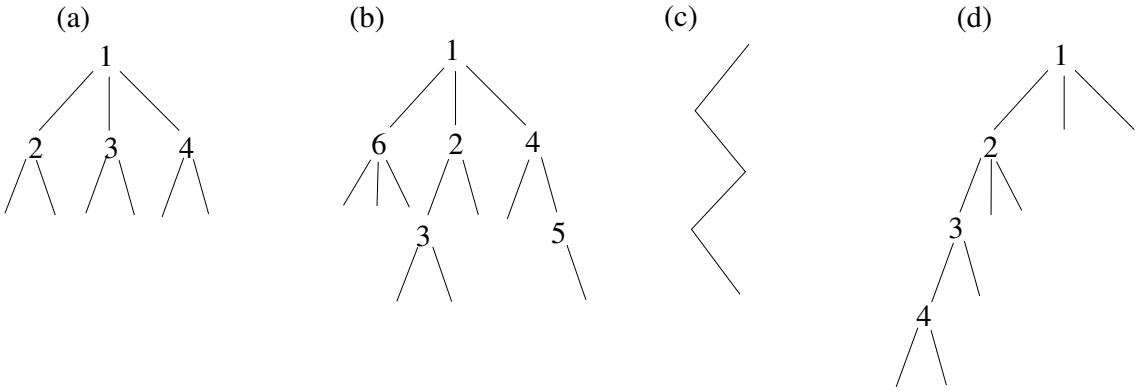
(b) How is DFS (or DLS) different from Backtrack?

(c) Which is more efficient if step costs are the same, BFS or UCS? Which is optimal if step costs are different?

(d) Modify the TreeSearch algorithm in Slide 32 of LN3, Part 1 to implement BFS.

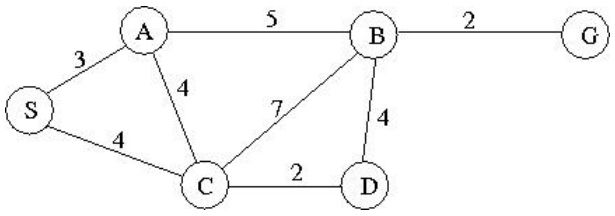
Exercise 5: Search Trees

Each of the following search structures has a distinctive shape and order of expansion which can be produced by a particular search procedure. For each tree write the name of the search procedure which can generate it — use the most specific option. The possible names are: backtrack, breadth first search, depth first search and tree search. Where applicable, the nodes are labeled with the order in which they are expanded.



Exercise 6: Search Algorithms

The diagram below depicts the cost of travelling between cities.



(a) Draw the search trajectory generated by the Backtrack procedure to reach the goal G starting from S, assuming that the BOUND for discontinuing the search is 4 steps. Use the heuristic *cheapest road* to order the rules. When backtracking, state clearly the reason for backtracking. What is the generated path? What is its cost?

(b) Draw the search *tree* generated by the Uniform-Cost Search (UCS) Treesearch algorithm to reach the goal G starting from S. What is the generated path? What is its cost?

(c) Draw the search *graph* generated by the UCS Graphsearch algorithm to reach the goal G starting from S. What is the generated path? What is its cost?